Sintaxis Java Procedural

AEDI / PROII

Grado en Ingeniería Informática

Sintaxis Java Procedural

Contenidos

1. Identificadores
2. Palabras clave
3. Tipos de datos de Java
4. Declaración de variables
5. Declaración de constantes
6. Conversión de tipos
7. Operadores
8. Expresiones
9. Sentencias
10. Arrays
11. String
12. Métodos
13. Entrada/Salida

1. Identificadores

- **Los *identificadores* son nombres que identifican inequívocamente cualquier elemento del lenguaje (variables locales, atributos, métodos, clases, …)**
- **Los identificadores deben comenzar por una letra, un subrayado (_) o un signo de dólar ($).**
- **Los identificadores pueden contener dígitos, pero no comenzar por ellos.**
- **Los identificadores no pueden contener espacios en blanco (tabuladores, espacios, avances de línea ni retornos de carro).**
- **Los identificadores pueden tener cualquier longitud.**
- **Java es sensible a las mayúsculas, por lo que los identificadores distinguen entre mayúsculas y minúsculas.**
- **Las palabras clave de Java no pueden emplearse como identificadores.**

Ejemplos válidos:

a1, Rectangulo20, getDistancia, $transaccion, _variable_sistema

Ejemplos no válidos:

1a, rectangulo 20, -variable

2. Palabras clave

- Las palabras clave son identificadores predefinidos que tienen un significado especial para el compilador de Java y que no pueden redefinirse.

abstract	class	final	instanceof	private	super	void
boolean	continue	finally	int	protected	switch	volatile
break	default	float	interface	public	syncronized	while
byte	do	for	long	return	this	
case	double	if	native	short	throw	
catch	else	implements	new	static	transient	
char	extends	import	package	strictfp	try	

- **true, false,** and **null** no son palabras claves pero son palabras reservadas, así que tampoco pueden ser utilizadas como identificadores.

3. Tipos de datos de JAVA

Java distingue entre:

- tipos de datos *primitivos*: están integrados en el sistema y no son objetos. Se clasifican en numéricos, booleanos y caracteres;
- tipos de datos *referenciados:* clases, interfaces y arrays.

✓ **Numéricos**

Tipo	**Tamaño**	**Descripción** (valores positivos menor y mayor)
byte	8 bits	entero muy pequeño con signo (- 128 ..127)
short	16 bits	entero corto con signo (- 32768 ..32767)
int	32 bits	entero con signo (- 2.14e+9 .. 2.14e+9)
long	64 bits	entero largo con signo (-9.22e+18 ..9.22e+18)
float	32 bits	número de coma flotante (1.402e-45 .. 3.402e+38)
double	64 bits	coma flotante de precisión doble (4.94e- 324 .. 1.79e+308)

3. Tipos de datos de JAVA

- **Booleanos**

Los tipos de datos booleanos tienen dos valores: **true** (verdadero) y **false** (falso).

Ejemplo:

```
int a = 1;
int b = 0;
boolean bool = a < b;     // la variable bool tiene el valor false
```

A diferencia de C/C++, boolean no es un tipo numérico y no se pueden realizar operaciones aritméticas con las variables boolean.

3. Tipos de datos de JAVA

- **Caracteres**

En Java, el tipo de datos **char** almacena un solo carácter Unicode. Por tanto, el tipo **char** de Java tiene 16 bits. Al igual que en C/C++, los caracteres se consideran enteros (en Java son enteros sin signo) de modo que pueden ser operandos de operaciones aritméticas.

Ejemplo:

```
char  letra = 'a';
letra = (char) ( letra – 32 ); // La variable letra tendrá el carácter 'A'
```

4. Declaración de variables

- Sintaxis:

  tipo_dato identificador [= valorInicial];

  Ejemplos:

```
int  p;
float  x;
float  y = 4.1;
float  z = 2.2;
boolean  endOfFile = false;
char  char1 = 'T';
```

- Las variables locales siempre se declaran al comienzo de cada bloque, delimitado por llaves (‘{‘ y ‘}’). Debe tratarse de inicializar las variables lo antes posible.

4. Declaración de variables

- Usando *var*

var identificador = valorInicial ;

- ✓ Permite declarar variables locales sin especificar el tipo de dato de forma explícita, dejando que el compilador infiera el tipo de dato según el valor asignado.
- ✓ Hace que el código sea más conciso cuando el tipo de dato es obvio.
- ✓ No se puede usar en atributos de clase, ni en parámetros de métodos ni en valores de retorno.
- ✓ Requiere inicialización inmediata para inferir correctamente el tipo de dato, por lo que no se puede declarar una variable con *var* sin asignarle un valor.
- ✓ Una vez inferido el tipo de dato, éste es fijo, no cambia.

Ejemplos con datos primitivos:

```
var i = 10;          // el compilador infiere el tipo int
var d = 2.2;         // el compilador infiere el tipo double
var b = false;       // el compilador infiere el tipo boolean
var c = 'a';         // el compilador infiere el tipo char
```

Al usar *var,* Java no se convierte en un lenguaje dinámico, sigue siendo fuertemente tipado.

5. Declaración de constantes

- Para declarar constantes se utiliza el calificador **final.**
- Sintaxis:

final tipo_dato nombreConstante = valor;

final var nombreConstante = valor;

(*var* solo se usa para declarar constantes locales y si se deduce claramente el tipo de dato por el valor asignado)

Ejemplo:

```
final double PI = 3.1416;
final var PI = 3.1416;
```

Una vez inicializada una constante, por definición no se le puede asignar otro valor.

6. Conversión de tipos

- En algunos casos, es necesario convertir una variable de un tipo en otro.
- Este proceso se denomina *conversión de tipo*. Para convertir el tipo de una variable, basta escribir el tipo de destino entre paréntesis inmediatamente delante del identificador de la variable.

Ejemplo:

```
byte  a;
int b=1;
int c=2;
a = b+c;
```

Error de compilación

```
a = (byte) (b+c);
```

Solución

6. Conversión de tipos

- La regla general es que el tipo convertido debe tener como mínimo un tamaño igual al tipo original. La siguiente tabla muestra las conversiones de tipo que no producen pérdida de información:

Tipo original	Tipo convertido
byte	short, char, int, long, float, double
short	int, long, float, double
char	int, long, float, double
int	long, float, double
long	float, double
float	double

7. Operadores

Operadores aritméticos

Operador	Definición	Precedencia	Asociatividad
++/--	Auto-incremento/decremento	1	Derecha
+/-	Más/menos unario	2	Derecha
*	Multiplicación	4	Izquierda
/	División	4	Izquierda
%	Modulo	4	Izquierda
+/-	Adición/sustracción	5	Izquierda

Ejemplos:

```
int y = 3;
int x;
int z;
x = ++y;
z = y--;
```

```
var suma = 5 + 3;          // int
var división = 10 / 3.0;   // double
var resto = 10 % 3;        // int
```

7. Operadores

Operadores lógicos

Ejemplo:

```
if (a < 3  &  j < 5) {
      h++;
}

if (a < 3  &&  j < 5) {
h++;
}
```

Operador	Definición	Precedencia	Asociatividad
!	Complemento lógico unario (NOT)	2	Derecha
&	Evaluación AND	9	Izquierda
^	XOR	10	Izquierda
|	Evaluación OR	11	Izquierda
&&	AND inmediato	12	Izquierda
||	OR inmediato	13	Izquierda

7. Operadores

Operadores de comparación

Operador	Definición	Precedencia	Asociatividad
<	Menor que	7	Izquierda
>	Mayor que	7	Izquierda
<=	Menor o igual que	7	Izquierda
>=	Mayor o igual que	7	Izquierda
= =	Igual	8	Izquierda
!=	Distinto	8	Izquierda

7. Operadores

Operadores de asignación

Ejemplos:

```
x = y = z = 1;

int y = 2;
y *= 2;   // equivale a (y = y * 2)

boolean b1 = true;
boolean b2 = false;
b1 &= b2;   // equivale a (b1 = b1 & b2)
```

Operador	Definición	Precedencia	Asociatividad
=	Asignación	15	Derecha
+=	Sumar y asignar	15	Derecha
-=	Restar y asignar	15	Derecha
*=	Multiplicar y asignar	15	Derecha
/=	Dividir y asignar	15	Derecha
&=	AND con asignación	15	Derecha
|=	OR con asignación	15	Derecha
^=	XOR con asignación	15	Derecha

7. Operadores

- **Operador condicional**

- Sintaxis:

expresion1 ? expresion2 : expresion3;

Se evalúa en primer lugar **expresion1**. Si el valor resultante es verdadero, se evalúa la **expresion2**; si es falso, se evalúa la **expresion3**.

Ejemplo:

```
int  x = 3;
int  y = 4;
int  max;
max = (x > y)? x : y;     /* max recibe el valor de x o de y, dependiendo de
   que x sea mayor que y. */
var maximo = (x > y) ? x : y; /* el compilador infiere el tipo de dato int a la
   variable maximo. var se usa correctamente en esta sentencia, ya que
   tanto expresion2 como expresion3 al evaluarse dan valores del mismo
   tipo y además es obvio dicho tipo. */
```

8. Expresiones

- Una expresión es una combinación de identificadores, palabras clave, símbolos y operadores que tiene sentido y que devuelve un valor.
- El tipo de dato del valor que devuelve una expresión depende de los elementos usados en la expresión.
- Hablando en términos generales, todo lo que pueda colocarse a la derecha de un signo de asignación es una expresión.

Ejemplos:

```
s = "Hola a todos";
x = 123.4;
y = (x * 5) / 2;
value = getValue();
```

9. Sentencias

- Las sentencias representan comandos para el compilador.
- Las sentencias se separan por punto y coma.
- Los *bloques de código* son grupos de sentencias que se comportan como una unidad.
- En Java, los bloques de código se delimitan mediante llaves ({ y }).
- Las definiciones de clases, los bucles, las sentencias condicionales, las sentencias de excepciones y el código principal de las funciones son ejemplos de bloques de código.

9. Sentencias

Sentencias de control.

Bucle while

- Sintaxis:

```
while (condición) {
        código para ejecutar en el bucle; // cuerpo del ciclo
}
```

En primer lugar, el bucle comprueba si se cumple la condición. En Java una condición debe devolver un valor booleano, en C/C++, puede devolver cualquier valor. Si el valor de la condición es verdadero, se ejecutan todas las sentencias del bloque. A continuación, se evalúa nuevamente la condición y se repite el proceso hasta que el resultado de la evaluación sea falso. En ese momento, el bucle finaliza su ejecución.

Ejemplo:

```
int  x = 0;
while ( x < 10 )  {
    System.out.println( "Bucle“ );
    x++;
}
```

9. Sentencias

Bucle do..while

- Sintaxis:

```
do {
        código para ejecutar en el bucle; // cuerpo del ciclo
 } while (condición) ;
```

El bucle **do** es similar al **while**, con la diferencia de que evalúa la condición después de las sentencias, no antes.

Ejemplo:

```
int   x = 0;
do{
   System.out.println( "Bucle“ );
   x++;
} while  ( x < 10 );
```

La principal diferencia entre las dos estructuras es que, **do.. while** siempre se ejecuta, como mínimo, una vez.

9. Sentencias

- **Bucle for**
  - Sintaxis:

```
for ( inicialización; condición; incremento ) {
  código para ejecutar en el bucle;
}
```

El bucle **for** consta de tres partes: una expresión de inicialización, una expresión de condición y una expresión de incremento.

Normalmente, la tercera expresión actualiza la variable del bucle inicializada en la primera expresión.

Ejemplo:

```
for ( int x = 0 ; x < 10 ; x++) {
   System.out.println( "Bucle“ );
}
```

9. Sentencias

Sentencia if..else

- Sintaxis:

```
if (condición1) {
 // bloqueCódigo 1
}
 else if (condición2) {
 // bloqueCódigo 2
}
else {
 // bloqueCódigo 3
}
```

Ejemplos:

```
if ( x % 2 == 0) {
      System.out.println("x es par");
}
else {
            System.out.println("x es impar");
}

if (x == y) {
      System.out.println("x es igual a y");
}
else if (x < y) {
      System.out.println("x es < que y");
}
else {
      System.out.println("x es > que y");
}
```

Por lo general, la sentencia **if-else** se compone de varios bloques. Solamente se ejecuta uno de los bloques cada vez que se ejecuta la sentencia **if-else** (naturalmente, esto depende de qué condición sea cierta). Los bloques **if-else** y **else** son optativos. Por otra parte, la sentencia **if-else** no está limitada a tres bloques: puede contener cuantos sean necesarios.

9. Sentencias

- **Sentencia switch** (clásico)
  - Sintaxis:

```
switch (expresión){
 case valor1: codeBlock1;
          break;
 case valor2: codeBlock2;
          break;
 default : codeBlock3;
 }
```

Ejemplo:

```
switch (c) {   // suponiendo que c sea de tipo char
  case '1': case '3': case '5': case '7': case '9':
            System.out.println( "c es un número impar“ );
            break;
  case '0': case '2': case '4': case '6': case '8':
            System.out.println( "c es un número par“ );
            break;
  case ' ':  System.out.println( "c es un espacio“ );
            break;
  default : System.out.println( "c no es un número ni un
espacio“ );
}
```

La sentencia **switch** presenta algunas características destacables:

- Los bloques de código no necesitan ir encerrados entre llaves.
- El bloque **default** corresponde al bloque **else** de la sentencia **if-else**.
- Los bloques de código se ejecutan o no dependiendo del valor de una expresión, no de una condición.
- El valor de la expresión ha de ser de tipo entero (o de un tipo que pueda convertirse sin problemas a **int**, como **char**) o *String*.
- Los valores **case** han de ser expresiones constantes del mismo tipo que el valor de la expresión.
- La palabra clave **break** es optativa. Finaliza la ejecución de una alternativa después de haber ejecutado su bloque. Si, por ejemplo, no se emplea después de *bloqueCódigo1* y éste se ejecuta, *bloqueCódigo2* se ejecuta inmediatamente después de él (lo que puede ser útil, pero en la mayoría de los casos no se desea).

9. Sentencias

- **Sentencia switch** (moderno, expresión)

Puede considerarse:

1. Una sentencia, si no se asigna a nada, que ejecuta las instrucciones de una alternativa case.
2. Una expresión que devuelve un valor si está a la derecha del operador de asignación. Sintaxis:

```
resultado = switch (expresión) {
        case valor1 -> resultado1;
        case valor2 -> resultado2;
        case valor3, varlo4 -> resultado3;
        default -> resultadoPorDefecto;
        };
```

El **default** es obligatorio:
1. Cuando **switch** se emplea como expresión
2. Cuando el compilador no puede deducir que todos los casos están cubiertos.

El operador flecha (->) elimina la necesidad de usar **break** y devolverá un valor que será el resultado de toda la expresión **switch.**

Ejemplo:

switch clásico

```
int días;
switch (mes) {
  case 1:
  case 3:
  case 5:
      días = 31;
      break;
   default:
      días = 30;
}
```

switch moderno

```
int días = switch (mes)
{
   case 1, 3, 5 -> 31;
   default -> 30;
};
```

9. Sentencias

- **Sentencia switch** (moderno, expresión)

  Si se usa un **switch** como una expresión y en una de sus alternativas **case** existen varias instrucciones, entonces desde ese bloque no se puede devolver un valor directamente con el operador flecha (->) sino que será necesario usar **yield**.

  Si se usa la palabra clave **yield** en la expresión ***switch***, todas las alternativas, incluyendo el caso **default** (si está presente), deben retornar un valor.

  - Sintaxis:

```
resultado = switch (expresión) {
 case valor1 -> {    // bloque de sentencias
                     yield resultado1;
 }
 case valor2 -> resultado2;
 case valor3, varlo4 -> {   // bloque de sentencias
                            yield resultado3;
 }
 default -> resultadoPorDefecto;
 }
```

Ejemplo válido:

```
int tipo = 2;
var descuento = switch ( tipo ) {
   case 1 -> 5;
   case 2 -> {   System.out.println( “Es el tipo de descuento”);
                 yield 10;
   }
   default -> {
      yield 0;
   }
};
```

Ejemplos no válidos:

```
var dias = switch (mes) {
   case 1, 3, 5 ->  {
                        return 31;
   }
};
```

```
var días = swtich (mes) {
     case 1 -> {
                    yield 31;
     }
     case 2 -> {
                    System.out.prinln (“30”);
      }
};
```

10. Arrays

- Un array es una estructura de datos con un tamaño fijo formada por elementos del mismo tipo de datos.

- Declaración

  tipo_dato [ ] nombre_array; int [ ] numeros;

- Creación

  nombre_array = **new** tipo_dato [tamaño]; numeros = new int [5];

- Declaración y creación

  tipo_dato [ ] nombre_array = **new** tipo_dato [tamaño];

  int [ ] numeros = new int [5];

- Acceso a elementos. (Se indexa desde 0.)

  nombre_array [índice] numeros [0] = 7;

- Inicialización directa

  tipo_dato [ ] nombre_array = { lista de valores separados por coma };

  int [ ] numeros = { 1, 2, 3, 4, 5 }; var numeros = new int [ ] { 1, 2, 3, 4, 5 };

  var numeros = { 1, 2, 3, 4, 5 }; // No compila

10. Arrays

- Recorrer un array unidimensional (vector)

```
int [ ] numeros = { 1, 2, 3, 4, 5 };
```

for clásico

```
for (int varControl = 0; varControl < nombre_array.length; varControl ++ ) {
        nombre_array [ varControl ]
}
```

```
for (int i = 0; i < numeros.length; i++ ) {
    numeros [ i ] = i * 3;
}
```

```
for (var i  = 0; i < números.length; i++ ) {
        números [ i ] = i * 3;
 }
```

for- each

```
for (tipo_dato elemento : nombre_array) {
        elemento
}
```

```
for (int valor : numeros) {
    valor = 3;
}
```

```
for (var valor : números ) {
        valor = 3;
}
```

11. String

- Un **String** es una secuencia de caracteres que representa texto.
- Los caracteres dentro de un **String** tienen un orden específico que es el que determina el significado del texto.

Ejemplo

```
String mensaje = “Hola mundo”;
System.out.print (mensaje); // visualiza por pantalla el texto Hola mundo
```

- Una vez creado un String, su contenido no se puede cambiar (Inmutable).

Ejemplo

```
String nombre = “Nanny”;
String apellidos = “González Rufino”;
String nombreCompleto = nombre + “  “ + apellidos;
                                    // usando el operador de concatenación (+)
System.out.println (nombreCompleto);
```

11. String

- Para asignar a un **String** texto que ocupa varias líneas se usa un *Text Block* (cadena multilínea). Un *Text Block* es un literal de cadena delimitado por tres comillas dobles (“””) que:
  - Preserva saltos de línea. No es necesario usar el carácter de barra invertida (\n) ni el operador de concatenación (+). Automáticamente cada línea incluirá un salto de línea al final.
  - Gestiona automáticamente la indentación. Aplica a todas las líneas la *indentación común mínima*, que consiste en recortar a la izquierda la cantidad de espacios en blanco que todas las líneas tienen en común.

Ejemplos:

Text Block	Equivalente	Salida	Observacion
String s = ””” línea 1 línea 2 ”””;	“línea 1 \nlínea 2\n”	línea 1 línea 2	Añade un salto de línea después de cada línea
String s = ””” línea 1 línea 2”””;	“línea 1 \nlínea 2”	línea 1 línea 2	Al cerrar en la última línea no se añade un salto de línea al final

String s = ”””
4 blancos
  10 blancos
4 blancos mas ”””;
		4 blancos
		  10 blancos
		4 blancos mas
			En la salida no incluye los 4 espacios iniciales comunes a todas las líneas

12. Métodos

- Lo que en otros lenguajes se conoce como función o procedimiento, en Java se llama *método*.
- La *cabecera de un método* consiste en un nombre, una lista de parámetros y el tipo del resultado.
- El código que realmente implementa el método, a veces llamado el *cuerpo del método*, es formalmente un bloque.
- La *declaración de un método* está formada por una cabecera y un cuerpo.

Ejemplo:

```
public static int minimo ( int x, int y )
{
   return   x < y ? x : y;
}
```

12. Métodos

- Colocando delante de un método las palabras reservadas *public static*, se puede imitar una función global del estilo de C.
- La sentencia **return** se usa para devolver un valor al punto de llamada. Si el tipo del resultado es *void* entonces no se devuelve ningún valor.
- Existe un método estático especial, el método *main*. Es el primer método que se llama cuando se ejecuta un programa.
- Sintaxis:

```
public static void main ( String  [ ]  args )
{
        // cuerpo del método
}
```

Es obligatorio el tipo del parámetro del main (String [ ]) y el tipo del resultado (void) sean el mostrado.

12. Métodos

- El nombre de un método es un identificador.
- La lista de parámetros consiste en cero o más *parámetros formales*, cada uno de ellos con un tipo de dato.
- Cuando se llama a un método, los *argumentos reales* se pasan a los parámetros formales usando asignación. Esto significa que los tipos primitivos se pasan siempre usando *paso de parámetros por valor*.
- Las declaraciones de los métodos pueden aparecer en cualquier orden.

Ejemplo:

```
public class PrimerEjemplo {

 public static int minimo (int x, int y)
 {
   return  x < y  ? x : y;
 }

 public static void main ( String [ ]  args )
 {
   int  a = 3:
   int  b = 7;
   System.out.println( minimo( a, b ) );
 }
}
```

12. Métodos

- Las variables declaradas dentro del cuerpo de un método son variables locales y solamente se pueden acceder a ellas a través de su nombre, dentro del cuerpo del método.

- Java permite la sobrecarga de nombres de los métodos. Esto significa que varios métodos pueden tener el mismo nombre mientras sus *signaturas* (es decir, tipos, número u orden de parámetros especificados en la lista) sean diferentes.

Ejemplo:

Supóngase que se necesita escribir una rutina que devuelva el valor máximo de una lista de parámetros. Una cabecera razonable para el método sería:

```
public static int maximo (int x, int y, int z)
```

y se podría tener también,

```
public static int maximo (int x, int y)
```

- Se puede observar que el tipo del resultado no está incluido en la signatura. Esto significa que no está permitido tener dos métodos cuya única diferencia sea el tipo del resultado.

13. Entrada/Salida

Salida

Cuando se ejecuta un programa Java, por defecto se crea el objeto *System.out* que permite imprimir una cadena de texto por pantalla usando su método *print*.

Si lo que se desea es incluir varias cadenas junto con el valor de diversas variables se usará el operador de concatenación **+**

Ejemplos:

```
System.out.print( “Mensaje que se visualiza.” ); /* las constantes de caracteres va entre comillas dobles */

int ultimo = 2;

System.out.print( “El primer impar es “ + 1 + “ y el último es “ + ultimo ); /* Se realiza la conversión a texto tanto del entero 1 como del valor almacenado en la variable ultimo */
```

Si se desea añadir una línea al final de la cadena visualizada se puede usar el carácter de barra invertida de salto de línea (\n) o el método *println*

Ejemplos:

```
System.out.println( “Mensaje que se visualiza.” );
System.out.print( “Mensaje que se visualiza.\n” );
```

13. Entrada/Salida

Salida (*Text Blocks*)

Si la cadena de salida ocupa varias líneas se usa *Text Blocks*.

Ejemplos:

```
System.out.println( """
Hola
Mundo
Java
""");
```

Salida

Hola
Mundo
Java

Java elimina los 4 espacios comunes a las 3 líneas. La segunda línea conservará los 6 espacios restantes.

```
System.out.println( """
    con 4 espacios
        con 10 espacios
    con 4 espacios también
""");
```

Salida

con 4 espacios
    con 10 espacios
con 4 espacios también

13. Entrada/Salida

Entrada

La clase *Scanner* permite realizar la lectura de datos. Para ello, se deben hacer los siguientes pasos:

1. Importar la clase *Scanner*:

   ```
   import java.util.Scanner; // La clase Scanner está definida en la librería java.util
   ```

2. Instanciar (crear) un objeto de esta clase y asociarle el dispositivo de entrada (objeto *System.in*):

   ```
   Scanner entrada = new Scanner ( System.in );  // entrada es el objeto creado de la clase Scanner
   ```

La clase *Scanner* ofrece varios métodos para leer datos del teclado:

nextLine : lee una cadena de caracteres (un objeto de la clase *String*)

```
String nombre;
nombre = entrada.nextLine();
```

nextInt : lee un número entero

```
int edad;
edad = entrada.nextInt();
```

nextFloat : lee un número decimal (real)

```
float nota;
nota = entrada.nextFloat();
```

13. Entrada/Salida

Entrada

Para evitar “problemas” cuando se mezcla la lectura de datos de distintos tipos, es preferible leer todos los datos del teclado como cadenas de texto (objetos de la clase String) y posteriormente convertirlas al tipo de dato que se requiera.

Ejemplo:

```
import java.util.Scanner;

public static void main(String[ ] args) {
  String nombre;
  int edad;
  float nota;

  Scanner entrada = new Scanner (System.in);

  System.out.print("Introduce nombre: ");
  nombre = entrada.nextLine();
  System.out.print("Introduce edad: ");
  edad = Integer.parseInt(entrada.nextLine()); // convierte una cadena de texto en un número entero
  System.out.print("Introduce nota: ");
  nota = Float.parseFloat(entrada.nextLine()); // convierte una cadena de texto en un número decimal
  System.out.println("El alumno " + nombre + " de " + edad + " años, ha obtenido " + nota + " puntos.");
}
```

```
String temp;

temp = entrada.nextLine();
edad = Integer.parseInt (temp);
```